Software Requirements Specification

Weather-Aware Truck Routing

React + Django web application

Document status:	Baseline requirements
Version:	1.0
Date:	28 October 2026
Prepared for:	Spotter.ai full-stack assessment

Purpose. Define the product behavior, business rules, interfaces, quality attributes, and acceptance criteria for selecting a safer truck route using forecast weather, load weight, and travel time.

::: titlepage **Software Requirements Specification**

Weather-Aware Truck Routing

React + Django web application

---------- ---------- **Document status:** Baseline requirements **Version:** 1.0 **Date:** 28 October 2026 **Prepared for:** Spotter.ai full-stack assessment **Primary audience:** Product, engineering, QA, and reviewers ---------- ----------

This document expands the supplied assessment brief. Where the brief is silent, decisions are stated as assumptions or recommendations and must be confirmed before production implementation. :::

Document Control

Revision history

::: tabularx p0.12p0.18Y Y **Version** & **Date** & **Author** & **Change summary** & 2026-10-28 & Spotter.ai & Initial detailed SRS derived from the supplied assessment brief. :::

Normative language

The terms **MUST**, **SHOULD**, and **COULD**indicate requirement priority. A **MUST**requirement is required for acceptance. A **SHOULD**requirement is important but may be deferred only with an approved product decision. A **COULD**requirement is an optional enhancement and is not required for the initial assessment.

Executive Summary

Weather-Aware Truck Routing is a decision-support web application for truck drivers and dispatchers. A user supplies an origin, destination, departure date/time, and load weight. The system obtains multiple route alternatives from a routing provider, estimates arrival time at checkpoints along each route, retrieves forecast weather for those locations and times, applies weather and load-specific risk rules, and recommends the best route.

The primary decision principle is safety first: minimize miles classified as Severe or No Travel before optimizing lower risk and duration. The UI presents all alternatives on an interactive map, checkpoint-level conditions and ETAs, route summaries, and a weather heatmap with a 0--48 hour forecast slider. The application is not a routing engine and does not replace driver judgment, official road closures, or a carrier's safety policy.

Success criteria

? A valid request produces three distinct route options when the routing

provider returns at least three alternatives.

? Every displayed checkpoint has a weather observation/forecast time

derived from the truck's estimated arrival time, not merely the trip start time.

? The risk classification and load rules are deterministic, auditable,

and covered by automated tests.

? The recommended route is reproducible from the displayed route metrics

and stated ranking rules.

? The map remains usable while route and weather data are loading, and

failures are visible with actionable recovery guidance.

? The complete assessment package includes a GitHub repository, hosted

application, and a 5--10 minute walkthrough.

Product Scope

In scope

? Geocoding origin and destination input.

? Truck trip request validation, including departure date/time and load weight.

? Retrieval and normalization of up to three route alternatives from a third-party routing API.

? Checkpoint generation at nominal 10, 25, or 50 mile spacing, including route endpoints.

? ETA calculation at each checkpoint using route distance and a provider-supplied or calculated travel duration.

? Weather forecast retrieval for checkpoint coordinates and ETA timestamps.

? Wind, rain, and snow risk classification using the supplied thresholds.

? Load-aware escalation to Severe or No Travel.

? Route scoring, recommendation, map display, checkpoint details, and route summaries.

? Corridor weather heatmap and a 0--48 hour forecast slider.

? Explicit loading, empty, degraded, and error states.

? Automated unit, integration, and acceptance tests for core logic.

Out of scope for the initial release

? Building or hosting a proprietary routing engine.

? Turn-by-turn navigation, dispatch, telematics, or vehicle control.

? Automatic road-closure, chain-law, bridge-clearance, fuel, or rest-stop optimization unless added as a separately approved integration.

? Guaranteed weather accuracy or safety certification.

? User accounts, saved trips, multi-vehicle fleets, and historical analytics.

? Automatic weather model selection without a documented provider policy.

Stakeholders and User Roles

::: tabularx p0.2Y Y **Role** & **Goals** & **Key needs** Truck driver & Choose a route with lower weather exposure before departure. & Fast inputs, clear risk labels, mobile-friendly map, prominent No Travel warnings. Dispatcher / fleet planner & Compare alternatives and communicate a defensible decision. & Route comparison, checkpoint detail, reproducible recommendation, shareable summary. Operations reviewer & Assess correctness and usability. & Visible assumptions, traceable metrics, stable demo data, reliable error handling. Product owner & Confirm business behavior and scope. & Acceptance criteria, prioritized requirements, quality measures. Engineering / QA & Implement and verify the system. & API contracts, data model, deterministic rules, test matrix. :::

System Context and Assumptions

Context

The browser is the primary client. A Django backend acts as the orchestration and policy layer. The backend calls geocoding, routing, and weather providers through isolated adapters. The frontend consumes a normalized application API and renders map and weather layers. Provider credentials **MUST**remain server-side.

Assumptions requiring confirmation

1. The routing provider can return at least three alternatives for

ordinary trips; if it cannot, the UI will show the number available rather than fabricate alternatives.

2. Distances are represented in statute miles and speeds in miles per

hour. Weather provider units are converted before risk evaluation.

3. Weather values are interpreted as forecast intensity over the

relevant checkpoint time window. The provider's exact field mapping must be documented.

4. A checkpoint's forecast timestamp is the estimated arrival timestamp

rounded or bucketed according to provider resolution.

5. If a route crosses more than the weather provider's forecast

horizon, the system marks unavailable checkpoints as Unknown and excludes them from false precision rather than silently using current weather.

6. The initial product uses a selected routing/weather provider

configuration and does not promise provider redundancy.

7. A recommendation is advisory. The UI must display a disclaimer that

conditions can change and official restrictions take precedence.

High-Level Architecture

Logical components

Component Responsibility Key constraints ---------- ---------- ---------- ---------- React web client Collect inputs, display map, route alternatives, checkpoint details, heatmap, and errors. Must not contain provider secrets or duplicate server-side risk policy. Django API Validate requests, orchestrate providers, calculate checkpoints/ETAs, apply risk rules, rank routes, and return normalized results. Must produce deterministic output for the same normalized inputs and provider payloads. Provider adapters Geocode, route, and retrieve weather data. Isolate vendor schemas, units, retries, timeouts, and attribution requirements. Risk engine Classify each weather metric and combine metric/load outcomes into checkpoint and route risk. Pure, unit-tested functions; no network or UI dependency. Recommendation engine Aggregate miles and risk metrics and apply the prescribed lexicographic ranking. Must expose the ranking rationale in the response. Map layer Render route polylines, checkpoint markers, risk styling, and heatmap. Must degrade gracefully if a visual layer cannot load.

Request lifecycle

1. Client validates required fields and submits a trip request.

2. Backend normalizes units, timestamps, coordinates, and route

preferences.

3. Backend geocodes text locations if coordinates are not supplied.

4. Routing adapter requests alternatives and normalizes geometry,

distance, duration, and legs.

5. Backend creates checkpoints and calculates checkpoint ETAs.

6. Weather adapter batches or parallelizes forecast requests within provider limits.

7. Risk engine classifies weather and applies load rules at each checkpoint.

8. Recommendation engine aggregates route metrics and sorts alternatives.

9. Backend returns a response containing all alternatives, the recommended route ID, checkpoint details, warnings, and metadata.

10. Client renders the result and permits route selection for inspection without changing the computed recommendation.

Functional Requirements

Trip input and validation

ID Requirement Priority ---------- ---------- ---------- **[FR-001]{style="color: blue"}** The system **MUST**provide fields for origin, destination, departure date/time, and load weight. Must **[FR-002]{style="color: blue"}** Origin and destination **MUST**accept human-readable addresses or place names and **SHOULD**support selecting geocoded suggestions. Must **[FR-003]{style="color: blue"}** Departure date/time **MUST**include an explicit timezone or derive one from the origin location and display the resolved timezone. Must **[FR-004]{style="color: blue"}** Load weight **MUST**be numeric, non-negative, and clearly labeled in pounds. The UI **SHOULD**provide a unit conversion affordance if other units are supported. Must **[FR-005]{style="color: blue"}** The client **MUST**block submission for missing, malformed, or contradictory inputs and identify each field requiring correction. Must **[FR-006]{style="color: blue"}** The backend **MUST**repeat validation and **MUST**not trust client-calculated distance, risk, ETA, or recommendation values. Must **[FR-007]{style="color: blue"}** The system **SHOULD**reject departure times outside the configured forecast horizon with a clear explanation, or proceed with explicit Unknown weather states where policy permits. Should **[FR-008]{style="color: blue"}** The user **SHOULD**be able to select checkpoint spacing of 10, 25, or 50 miles. The default **SHOULD**be 25 miles to balance detail and provider usage. Should

Geocoding and routing

ID Requirement Priority ----------

---------- **[FR-010]{style="color: blue"}** The system **MUST**resolve both endpoints to latitude/longitude coordinates before routing. Must **[FR-011]{style="color: blue"}** The system **MUST**request alternatives from a routing provider and return up to three viable route options. Must **[FR-012]{style="color: blue"}** Each route **MUST**include a stable route ID, display name, geometry, total distance, estimated duration, and provider metadata needed for attribution. Must **[FR-013]{style="color: blue"}** If fewer than three alternatives are returned, the response **MUST**state the number available and the UI **MUST**not imply that missing routes were evaluated. Must **[FR-014]{style="color: blue"}** The backend **MUST**normalize route distances to miles and durations to seconds before calculations. Must **[FR-015]{style="color: blue"}** Provider failures, no-route results, quota errors, and malformed payloads **MUST**be mapped to explicit application errors with a correlation ID. Must

Checkpoint and ETA processing

ID Requirement Priority ----------

---------- **[FR-020]{style="color: blue"}** The system **MUST**generate checkpoints along each route at the configured nominal spacing and include origin and destination checkpoints. Must **[FR-021]{style="color: blue"}** Checkpoints **MUST**be ordered by route progress and include route distance from origin in miles and coordinates. Must **[FR-022]{style="color: blue"}** The system **MUST**calculate an ETA for every checkpoint using the departure timestamp plus elapsed route travel time to that checkpoint. Must **[FR-023]{style="color: blue"}** ETA calculation **MUST**use route progress and route duration, not straight-line distance. For multi-leg routes, leg durations **SHOULD**be used when available. Must **[FR-024]{style="color: blue"}** The system **MUST**preserve the ETA timezone and return an ISO 8601 timestamp. Must **[FR-025]{style="color: blue"}** The response **MUST**identify whether an ETA is exact, interpolated, or unavailable. Should

Weather retrieval and normalization

ID Requirement Priority ----------

[FR-030]{style="color: blue"} The system **MUST**request weather for each checkpoint using the checkpoint coordinates and estimated arrival time. Must **[FR-031]{style="color: blue"}** Weather data **MUST**include, when available, wind speed, rain rate, snow rate, forecast timestamp, source, and unit metadata. Must **[FR-032]{style="color: blue"}** The backend **MUST**convert provider values into mph and inches per hour before classification. Must **[FR-033]{style="color: blue"}** Weather requests **SHOULD**be batched or cached where provider policy permits, while preserving checkpoint-specific timestamps. Should **[FR-034]{style="color: blue"}** Missing, stale, or out-of-horizon weather **MUST**be represented as Unknown with a visible warning; it **MUST**not be treated as Low risk. Must **[FR-035]{style="color: blue"}** The UI **MUST**show weather source and forecast time for selected checkpoints. Must

Risk and load evaluation

ID Requirement Priority ----------

[FR-040]{style="color: blue"} The risk engine **MUST**classify wind, rain, and snow using the exact threshold tables in Section 7. Must **[FR-041]{style="color: blue"}** A checkpoint's base weather risk **MUST**be the highest applicable classification across available weather metrics. Must **[FR-042]{style="color: blue"}** Load rules **MUST**be applied after base weather classification and **MUST**be visible in the checkpoint explanation. Must **[FR-043]{style="color: blue"}** Wind at or above 55 mph **MUST**result in No Travel for every load weight. Must **[FR-044]{style="color: blue"}** Wind from 45 through 54 mph with load weight greater than 30,000 lb **MUST**result in No Travel. Must **[FR-045]{style="color: blue"}** Wind from 35 through 44 mph with load weight greater than 40,000 lb **MUST**result in Severe. Must **[FR-046]{style="color: blue"}** The system **MUST**retain the triggering metric, threshold, and load rule in a machine-readable and human-readable explanation. Must **[FR-047]{style="color: blue"}** Unknown weather **MUST**not be silently downgraded. Route summaries **MUST**show Unknown miles or an explicit policy warning. Must

Recommendation and route summaries

ID Requirement Priority ---------- ----------

---------- **[FR-050]{style="color: blue"}** The system **MUST**aggregate each route's miles by risk level and calculate average risk using a documented numeric mapping. Must **[FR-051]{style="color: blue"}** Routes **MUST**be ranked lexicographically by: (1) fewest Severe miles, (2) fewest High miles, (3) lowest average risk, and (4) shortest travel time. Must **[FR-052]{style="color: blue"}** No Travel exposure **MUST**be surfaced as a blocking safety warning and **SHOULD**be ranked worse than any route without No Travel exposure. This policy **MUST**be documented in the response. Must **[FR-053]{style="color: blue"}** The response **MUST**include the recommended route ID, ranking position, and rationale values used for every comparison. Must **[FR-054]{style="color: blue"}** The UI **MUST**display a summary for each route including duration, distance, Severe/High/Moderate/Low/Unknown miles, and overall risk. Must **[FR-055]{style="color: blue"}** The user **MUST**be able to inspect any route without losing the recommended-route indicator. Must **[FR-056]{style="color: blue"}** Ties after all ranking dimensions **MUST**be resolved deterministically by stable route ID. Should

Map, heatmap, and interaction

ID Requirement Priority ----------

[FR-060]{style="color: blue"} The map **MUST**show all returned route geometries with visually distinct styling and a clear selected/recommended state. Must **[FR-061]{style="color: blue"}** The map **MUST**show checkpoint markers with risk-aware colors and a legend. Must **[FR-062]{style="color: blue"}** Selecting a route or checkpoint **MUST**show its details, including ETA, forecast values, classification, and rule explanation. Must **[FR-063]{style="color: blue"}** The map **MUST**show a weather heatmap or equivalent corridor intensity layer. Must **[FR-064]{style="color: blue"}** The UI **MUST**provide a 0--48 hour forecast slider relative to the departure time and update the heatmap state. Must **[FR-065]{style="color: blue"}** The slider **MUST**display its effective timestamp and **MUST**not imply unavailable forecast precision. Must **[FR-066]{style="color: blue"}** The UI **MUST**remain responsive during route/weather loading and show progress by phase where practical. Must **[FR-067]{style="color: blue"}** Required map attribution and provider notices **MUST**be visible as required by provider terms. Must

Deliverables and walkthrough

ID Requirement Priority ----------

---------- **[FR-070]{style="color: blue"}** The project **MUST**be deliverable as a GitHub repository with setup instructions, environment configuration guidance, tests, and documented provider dependencies. Must **[FR-071]{style="color: blue"}** A hosted

application **MUST**be available for evaluation with a safe demo configuration and no exposed secrets. Must **[FR-072]{style="color: blue"}** A 5--10 minute walkthrough **MUST**demonstrate input, route comparison, checkpoint details, recommendation logic, heatmap slider, and an error or limitation state. Must

Business Rules and Risk Model

Weather thresholds

Values are inclusive at the lower bound of each band and exclusive at the upper bound unless the table explicitly says otherwise. Values are normalized before comparison.

Condition Low Moderate High Severe / No Travel ---------- ---------- ---------- ---------- ---------- Wind <25 mph 25--34 mph 35--44 mph 45--54 mph Severe; $geq55$ mph No Travel Rain <0.10 in/hr 0.10--0.25 in/hr 0.25--0.50 in/hr 0.50--1.00 in/hr Severe; >1.00 in/hr No Travel Snow <0.5 in/hr 0.5--1.0 in/hr 1.0--2.0 in/hr 2.0--3.0 in/hr Severe; >3.0 in/hr No Travel

Boundary policy. To remove ambiguity in implementation, the following boundaries are recommended: 25 mph is Moderate; 35 mph is High; 45 mph is Severe; 55 mph is No Travel. For precipitation, 0.10, 0.25, 0.50, and 1.00 in/hr are assigned to the higher named band, while values above 1.00 are No Travel. Snow uses 0.5, 1.0, 2.0, and 3.0 in/hr in the same pattern, with values above 3.0 as No Travel. The product owner must approve this interpretation before release.

Load rules

? Wind $geq55$ mph: No Travel for any load.

? Wind 45--54 mph and load $>30,000$ lb: No Travel.

? Wind 35--44 mph and load $>40,000$ lb: Severe.

? The stated load rules do not reduce a classification. They only

escalate risk.

? Weight exactly 30,000 lb does not trigger the $>30,000$ rule; exactly

40,000 lb does not trigger the $>40,000$ rule.

Checkpoint classification

For each checkpoint, the engine evaluates each available condition independently, applies load escalation, and chooses the maximum severity using the ordered scale: $$text{Low} < text{Moderate} < text{High} < text{Severe} < text{No Travel}.$$ If one metric is No Travel, the checkpoint is No Travel. If a weather metric is unavailable, the checkpoint records Unknown for that metric and includes a data-quality warning. Unknown handling in route ranking must be a product decision; the recommended default is to rank routes with Unknown exposure below equivalent fully observed routes and to prevent a confident "safe" label.

Route aggregation and ranking

The system **MUST**return these per-route fields:

? Total distance and duration.

? Miles in each risk band, calculated from route segments between

checkpoints. Segment risk applies to the segment ending at the checkpoint unless a more detailed provider time series is used.

? Average risk score using the documented mapping Low=0, Moderate=1,

High=2, Severe=3, No Travel=4. Unknown is reported separately and is not silently assigned a score.

? A ranking tuple:

$(text{No Travel policy flag}, text{Severe miles}, text{High miles}, text{average score}, text{duration seconds}, text{route ID})$.

The No Travel policy flag is recommended as a leading safety gate even though the supplied preference list names Severe first. This is necessary to avoid recommending a route containing a prohibited segment. If the assessment requires literal ordering without a No Travel gate, that decision must be explicitly approved and tested.

User Experience Requirements

Primary screen

1. Header with product name, safety disclaimer, and provider attribution.
2. Trip form with origin, destination, departure date/time, load weight, spacing selector, and Calculate Routes action.
3. Map occupying the primary visual area.
4. Route comparison panel showing recommendation and alternatives.
5. Checkpoint detail panel or drawer opened by marker/route selection.
6. Forecast slider, timestamp, legend, and heatmap controls.

Visual semantics

State Required treatment ---------- ---------- Low Green label and non-alarming marker. Moderate Amber label with caution wording. High Orange label and prominent caution. Severe Red label, strong warning, and route-summary count. No Travel Dark red or equivalent high-contrast label, blocking warning, and explicit reason. Unknown Gray or patterned label; never styled as Low. Loading Skeleton/progress state that preserves the form and explains the current phase. Error Human-readable message, correlation ID where useful, and retry or correction action.

Accessibility

? Risk cannot be communicated by color alone; labels, icons, or text must accompany color.

? Keyboard users must reach all form controls, route selection controls, slider, and checkpoint details.

? Focus states must be visible and the map must have a textual route/checkpoint alternative.

? Text and controls must meet WCAG 2.1 AA contrast targets where applicable.

? Screen-reader labels must include route name, risk level, and the reason for No Travel.

External Interfaces and API Contracts

Provider interfaces

::: tabularx p0.2p0.26Y **Interface** & **Input** & **Output / handling** Geocoding & Query text, locale, optional bias & Coordinate, formatted label, confidence, provider ID. Cache successful results subject to terms. Routing & Origin/destination coordinates, alternatives flag, truck-relevant options where supported & Route geometry, distance, duration, legs, route ID, warnings, attribution. Weather & Latitude, longitude, forecast timestamp(s) & Wind, rain, snow, timestamp, units, source, quality/horizon metadata. :::

Application API

POST /api/v1/trips/route-analysis

Request (illustrative):

{ "origin": {"query": "Denver, CO"}, "destination": {"query": "Chicago, IL"}, "departure_at": "2026-11-02T08:00:00-07:00", "load_weight_lb": 42000, "checkpoint_spacing_miles": 25 }

Response (illustrative shape):

{ "request_id": "uuid", "recommended_route_id": "route-2", "routes": [{ "route_id": "route-2", "distance_miles": 1001.2, "duration_seconds": 55800, "risk_summary": { "no_travel_miles": 0, "severe_miles": 12.5, "high_miles": 68.0, "moderate_miles": 160.0, "low_miles": 760.7, "unknown_miles": 0, "average_risk_score": 0.41 }, "ranking": {"tuple": [0, 12.5, 68.0, 0.41, 55800, "route-2"]}, "checkpoints": [{ "checkpoint_id": "route-2-cp-04", "distance_from_origin_miles": 100, "latitude": 39.7, "longitude": -104.8, "eta": "2026-11-02T09:52:00-07:00", "weather": {"wind_mph": 38, "rain_in_hr": 0, "snow_in_hr": 0}, "risk": {"level": "SEVERE", "reasons": ["wind", "load_rule"]} }] }], "warnings": [], "provider_metadata": {"routing_source": "...", "weather_source": "..."} }

Error contract

Errors **MUST**use a stable code, safe message, request ID, and optional field/path details:

{ "error": { "code": "ROUTING_PROVIDER_UNAVAILABLE", "message": "Routes are temporarily unavailable. Please retry.", "request_id": "uuid", "retryable": true, "details": [] } }

Provider response bodies and credentials must not be returned to the browser or written to logs.

Data Model

::: tabularx p0.2p0.31Y **Entity** & **Required fields** & **Notes** TripRequest & request ID, origin, destination, departure timestamp, timezone, load weight, spacing & Immutable normalized input for one analysis. Route & route ID, geometry, distance, duration, provider metadata & One alternative under a TripRequest. Checkpoint & checkpoint ID, route ID, sequence, progress miles, coordinates, ETA, ETA quality & Includes endpoints. WeatherSnapshot & checkpoint ID, forecast timestamp, normalized metrics, source, quality & Must preserve raw/provider timestamp metadata where allowed. RiskAssessment & checkpoint ID, metric levels, overall level, reasons, load rules applied & Purely derived and reproducible. RouteSummary & route ID, miles by risk, average score, ranking tuple, recommendation flag & Derived from checkpoints/segments. :::

Non-Functional Requirements

ID Requirement Priority ----------

---------- **[NFR-001]{style="color: blue"}** For a normal trip and responsive providers, the first meaningful result **SHOULD**render within 10 seconds at the 95th percentile; the UI **MUST**show progress before then. Should **[NFR-002]{style="color: blue"}** Core risk and ranking calculations **MUST**be deterministic and independent of browser locale. Must **[NFR-003]{style="color: blue"}** The API **MUST**enforce provider timeouts, bounded retries with backoff, and concurrency limits. Must **[NFR-004]{style="color: blue"}** The client **MUST**avoid blocking the main thread while rendering routes, markers, and heatmap data. Must **[NFR-005]{style="color: blue"}** The application **MUST**not expose provider keys, secrets, or unrestricted internal errors to clients. Must **[NFR-006]{style="color: blue"}** Inputs **MUST**be validated and provider outputs schema-checked before use. Must **[NFR-007]{style="color: blue"}** Logs **MUST**include request IDs and phase/error codes but must exclude full addresses, secrets, and unnecessary personal data. Must **[NFR-008]{style="color: blue"}** The system **MUST**clearly identify forecast limitations, provider attribution, and advisory status. Must **[NFR-009]{style="color: blue"}** The application **SHOULD**support current Chrome, Firefox, Safari, and Edge versions used by the target audience. Should **[NFR-010]{style="color: blue"}** The map and results layout **MUST**be usable at mobile widths without hiding safety warnings. Must **[NFR-011]{style="color: blue"}** Test coverage **MUST**include all threshold boundaries, load boundaries, missing data, provider failures, and ranking ties. Must **[NFR-012]{style="color: blue"}** Deployments **MUST**use environment-based configuration and a documented health/readiness check. Must

Security, Privacy, and Reliability

? API credentials **MUST**be stored in deployment secrets or environment

variables and rotated without code changes.

? Origin/destination data **MUST**be treated as potentially sensitive

operational data. Do not retain it by default beyond the request lifecycle unless a retention policy is approved.

? Rate limiting **SHOULD**protect the analysis endpoint from accidental

or abusive high-volume use.

? Provider calls **MUST**use HTTPS and validate TLS in deployed environments.

? CORS, CSRF, and authentication posture **MUST**match the deployment architecture; unsafe wildcard production defaults are prohibited.

? The backend **MUST**fail closed for malformed provider data: do not produce a reassuring recommendation from an incomplete result.

? Retry behavior must avoid multiplying provider load and must not retry non-retryable validation errors.

Testing and Acceptance

Test levels

::: tabularx p0.2Y **Level** & **Coverage** Unit & Threshold boundaries, unit conversion, load escalation, checkpoint interpolation, aggregation, ranking, stable tie-breaking. Integration & Adapter normalization, provider error mapping, endpoint validation, response schema, cache behavior. Component/UI & Form validation, loading/error states, route selection, checkpoint details, slider/heatmap synchronization, accessibility. End-to-end & Submit representative trip, inspect three routes, verify recommendation rationale, trigger provider/degraded state, complete walkthrough path. Performance & Route/weather fan-out, map rendering, slider interaction, 95th percentile time-to-result target. :::

Acceptance criteria

ID Given / when / then Evidence ---------- ---------- ---------- ---------- ---------- **[AC-001]{style="color: blue"}** Given valid origin, destination, departure, and load; when the user submits; then the system displays up to three route options and their summaries. E2E test + screen capture **[AC-002]{style="color: blue"}** Given a route with a checkpoint at 35 mph wind; when load is 42,000 lb; then that checkpoint is Severe with a load-rule explanation. Unit test **[AC-003]{style="color: blue"}** Given a route with 50 mph wind and load 31,000 lb; then the checkpoint is No Travel. Unit test **[AC-004]{style="color: blue"}** Given a route with 55 mph wind and any load; then the checkpoint is No Travel. Unit test **[AC-005]{style="color: blue"}** Given a 1.01 in/hr rain value; then the checkpoint is No Travel. Unit test **[AC-006]{style="color: blue"}** Given three routes; when route metrics differ; then ranking follows Severe miles, High miles, average risk, duration, in that order. Unit/integration test **[AC-007]{style="color: blue"}** Given checkpoint weather; then its forecast timestamp equals or is traceably derived from the checkpoint ETA. Integration test **[AC-008]{style="color: blue"}** Given unavailable weather; then the UI shows Unknown/warning and does not label the route Low risk by default. Component/E2E test **[AC-009]{style="color: blue"}** Given a selected route; then its geometry, checkpoints, risk labels, ETAs, and summary are visible on the map/panel. E2E test **[AC-010]{style="color: blue"}** Given the forecast slider moves from 0 to 48 hours; then the heatmap timestamp and rendered layer update without a page reload. Component/E2E test **[AC-011]{style="color: blue"}** Given a routing provider error; then the API returns the stable error contract and the UI offers a retry or correction action. Integration/E2E test **[AC-012]{style="color: blue"}** Given production-like configuration; then no provider key appears in client bundles, logs, or API responses. Deployment/security check

Required boundary test matrix

Rule Boundary values Expected verification ---------- ---------- ---------- Wind 24.99, 25, 34.99, 35, 44.99, 45, 54.99, 55 mph Low/Moderate/High/Severe/No Travel transitions Rain 0.09, 0.10, 0.24, 0.25, 0.49, 0.50, 1.00, 1.01 in/hr Correct inclusive/exclusive interpretation Snow 0.49, 0.50, 0.99, 1.00, 1.99, 2.00, 3.00, 3.01 in/hr Correct inclusive/exclusive interpretation Load 30,000; 30,000.01; 40,000; 40,000.01 lb Strict greater-than load rules

Observability and Operations

? Each request receives a correlation/request ID returned in the response and shown in user-facing error details when appropriate.

? Metrics **SHOULD**include request count, provider latency/error rate,

weather coverage ratio, route count, analysis duration, and cache hit rate.

? Health checks **MUST**distinguish process health from provider readiness.

? Alerts **SHOULD**cover sustained provider failures, elevated latency, and unusually high Unknown-weather ratios.

? A feature flag or configuration switch **SHOULD**allow disabling a provider integration without changing the risk engine.

Traceability to Assessment Brief

::: tabularx p0.35Y **Assessment statement** & **Covered by** User enters origin, destination, departure date/time, load weight & FR-001--FR-008; Section 5.1 Generate three route options using routing API & FR-010--FR-015; Section 4 architecture Sample weather every 10/25/50 miles at estimated arrival time & FR-020--FR-035; Section 7.3 Calculate weather risk using risk/load rules & FR-040--FR-047; Section 7 Recommend best route balancing risk and travel time & FR-050--FR-056; Section 7.5 Show routes, checkpoints, risk levels, ETAs, summaries on map & FR-060--FR-062; Section 6 Heatmap with 0--48 hour forecast slider & FR-063--FR-067; Section 6 GitHub repository, hosted app, 5--10 minute walkthrough & FR-070--FR-072; Section 9 Evaluation weighting: logic, UI/performance, code quality, docs & Sections 10--13 and acceptance matrix :::

Risks and Mitigations

::: tabularx p0.23p0.27Y **Risk** & **Impact** & **Mitigation** Provider quota or outage & No route/weather result & Timeouts, bounded retries, visible degraded states, provider adapters, demo fixtures. Forecast horizon ends before destination & False confidence & Unknown classification, coverage warning, no Low fallback. Ambiguous threshold boundaries & Inconsistent recommendations & Approve boundary policy; table-driven tests at every boundary. Large route/checkpoint count & Slow UI and provider cost & Spacing selector, batching, caching, progressive rendering, request limits. Map library/rendering bottleneck & Poor evaluation UX & Simplified geometry, viewport fitting, marker clustering where needed, performance testing. Recommendation misunderstood as guarantee & Safety/operational harm & Persistent advisory disclaimer, reasons, source/time display, No Travel prominence. :::

Implementation Priorities

1. **P0:** Input validation, routing integration, checkpoint/ETA pipeline, risk engine, ranking, and deterministic tests.
2. **P0:** Core map with route lines, markers, summaries, selected route, and recommendation explanation.
3. **P1:** Weather heatmap, 0--48 hour slider, provider attribution, robust loading/degraded states.
4. **P1:** Accessibility, responsive layout, observability, deployment hardening, and walkthrough polish.
5. **P2:** Saved trips, authenticated fleet workflows, provider redundancy, road restrictions, richer operational overlays.

Open Decisions Before Production

1. Approve inclusive/exclusive interpretations for all threshold boundaries.
2. Confirm whether No Travel is a hard route-ranking gate ahead of the four supplied preference dimensions.

3. Define Unknown-weather ranking and whether a route with any Unknown segment may be recommended.

4. Select routing, geocoding, weather, and map providers; confirm quotas, terms, attribution, and forecast resolution.

5. Confirm expected truck-specific routing constraints supported by the selected provider.

6. Confirm data retention, authentication, rate limiting, and deployment region requirements.

7. Confirm the exact numeric average-risk policy and whether No Travel participates in the average.

Definition of Done

The release is complete when:

? All Must requirements and P0 priorities are implemented.

? Automated tests pass for the boundary matrix and acceptance criteria.

? A hosted deployment can perform a representative analysis without exposing secrets.

? The UI visibly explains recommendation rationale, forecast timestamps, Unknown data, and No Travel conditions.

? Repository documentation covers local setup, environment variables, provider configuration, test commands, and known limitations.

? The walkthrough demonstrates the complete happy path and at least one failure/degraded path within 5--10 minutes.

? Product owner approves the open decisions that affect classification and ranking behavior.